

Name: _____

Class: _____

Mr. Rawson • Computer Science



Year 10 2026

Compression: Lossy or Lossless?

You have calculated two file sizes, and both were large. An 800×600 photograph in true colour came to **1,440 kB**. Three minutes of CD-quality music came to **15,876,000 bytes**, nearly 16 MB. Today is about making files like these smaller, and the two ways of doing it.

By the end of this sheet you can:

- explain **the need for compression**, using common scenarios where it is needed
- describe the two **types of compression**, **lossy** and **lossless**, and the **effects on the file** of each
- give the **advantages and disadvantages of each type**

LO1 • Why compress?

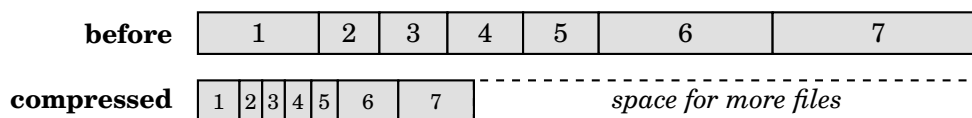
OCR 1.2.5 — the need for compression • common scenarios where it is needed

Key concept

Compression means making a file smaller.

Why we do it:

- files take up **less storage space**
- they **download and upload faster**
- they use **less bandwidth**, which matters on a mobile connection
- email and messaging services impose **size limits**



Worked example

Worked example: the song from Booklet 2. Uncompressed, it is 15,876,000 bytes, which is about **15.9 MB**. A music service compresses it to roughly one tenth of that, about **1.6 MB**.

A phone with 32 GB free, which is 32,000 MB, holds

- uncompressed: $32,000 \div 15.9 \approx$ **2,000 songs**
- compressed: $32,000 \div 1.6 =$ **20,000 songs**

The file is ten times smaller, so ten times as many songs fit, and each one downloads ten times faster.

(a) You have 10 videos of you and your friends, each 4 GB, and a 16 GB USB memory stick.

[2]

(i) How many of the videos fit on the stick? _____

(ii) Compressed, each video is 1.5 GB. How much space do all 10 now need? _____ GB

Check: your answer to (ii) is less than 16, so every video fits.

- (b) Mia wants to email 20 of these photographs, uncompressed, to her grandmother. Her email service accepts attachments up to 25 MB in total. Explain, using a calculation, why compressing the photographs solves her problem.

[3]

LO2 • Lossless: nothing is lost

OCR 1.2.5 — types of compression: lossless · effects on the file

There are exactly two kinds of compression, and the difference is whether anything is thrown away.

Key concept

Lossless — **nothing is lost**. The file is rewritten more efficiently, and decompressing gives you back the original *exactly*, bit for bit.

Say it out loud. Here is a row of fruit. Read it aloud exactly as it appears.



“Apple, apple, apple, pear, pear, banana, orange, orange, apple.” **9 fruit names**

Now group each **run** of the same fruit and say how many there are.



“Three apples, two pears, one banana, two oranges, one apple.” **5 counts**

Nothing has been left out. Anyone listening could lay the fruit back out in exactly the same order. The same information has been **written more efficiently**, and that is lossless compression.

The same idea in an image. One row of pixels from a black-and-white picture is 16 values. Stored as runs, it is 3 counts and 3 colours.



Booklet 2 page 14 does the same with letters: AAAABBBCCD becomes 4A3B2C1D. Storing a run as a count and a value is called **run-length encoding**. It is one way of doing lossless compression, and the idea that matters for the exam is the one underneath it: **the original comes back exactly**.

- (c) Rebuild this picture. Each row is stored as runs, read left to right: 2W means 2 white pixels, 1B means 1 black pixel. Shade the black pixels.

[4]

2W 1B 5W 1B 2W							
3W 1B 3W 1B 3W							
2W 7B 2W							
1W 2B 1W 3B 1W 2B 1W							
11B							
1B 1W 7B 1W 1B							
1B 1W 1B 5W 1B 1W 1B							
3W 2B 1W 2B 3W							

Check: every row adds up to 11 pixels, and your picture is the same on its left and right halves.

- (d) The picture you rebuilt is exactly the same as the original, pixel for pixel. Explain why this makes it an example of **lossless** compression.

[2]

Try it: the Text Compression Widget. Open the link on the class page, or go to studio.code.org/courses/text-compression/units/1/lessons/1/levels/2. Type a repeated word or phrase into the **dictionary**, and the widget swaps every copy of it in the text for a single symbol. It counts every character as one byte, and adds the dictionary to the total, because the dictionary is sent too.

- (e) Find as many repeated patterns as you can. Record your best result.

[2]

Original text (bytes)	Compressed text + dictionary (bytes)	Compression (%)

Check: your middle column is smaller than your first, so the widget shows a compression above 0%.

- (f) The dictionary is sent along with the compressed text. Explain why.

[2]

Effects on the file. The file is **smaller**, and its **quality is unchanged**: decompress it and you have the original, exactly. Plenty of files shrink a long way this way. A plain text document zipped with lossless compression is often less than half its original size.

Where it is used. Anything that must come back exactly as it was: text documents, program code and executable files, spreadsheets, ZIP files, and PNG images.

- (g) A program file is compressed so it can be downloaded. Explain why it is compressed with lossless compression.

[2]

LO3 • Lossy: much smaller, and the detail is gone for good

OCR 1.2.5 — types of compression: lossy · effects on the file

Key concept

Lossy — **data is permanently removed**. Detail the human eye or ear is unlikely to notice is discarded. You can never get it back.

See it. To make this photograph of an iguana smaller, the number of pixels has been cut right down. It is cut hard here so you can see exactly what was thrown away.



Original: $3,283 \times 2,188$ pixels
21,549,612 bytes, about **21.5 MB**



Compressed: 82×55 pixels
? kB (you work it out in (h))

The detail in the scales is gone, and no program can bring it back: the compressed file simply does not contain it. Everyday lossy compression, such as saving a photo as a JPEG, is far gentler. It removes detail you are unlikely to notice, so at normal size the photo looks the same.

Effects on the file. The file is **much smaller**, often about a tenth of the original, and its **quality is reduced**. The fine detail has been thrown away, so decompressing gives a copy that looks or sounds nearly the same, and never exactly the same.

Where it is used. Photos (JPEG), music (MP3, OGG) and video, especially video that is streamed. These are files a person looks at or listens to, where the eye and ear fill in the missing detail.

So why not always use lossless? Two reasons. Sound can rarely be made much smaller with lossless compression, and lossy compression can usually make a file far smaller than lossless can.

Lossless has its own formats for the same kinds of file: **PNG** for images and **FLAC** for sound. They are chosen when every detail has to be kept.

(h) The compressed iguana is 82×55 pixels, with a colour depth of 24 bits. Calculate its file size in kilobytes. Show your working.

[3]

Check: the original was over 21,000 kB, and your answer is well under 100 kB.

(i) Lossy compression permanently removes data from a photograph, yet most people cannot see any difference. Explain why.

[2]

(j) Complete the table. It is the whole topic in eight boxes.

[4]

	Lossless	Lossy
Effect on the file		
Original recoverable?		
Advantage		
Disadvantage		

Check: every box in the Lossy column says something about size or quality, and matches the table on page 13 of Booklet 2.